

FUNCIONES DE AGREGADO Y AGRUPAMIENTO

CÁLCULOS SOBRE CONJUNTOS DE REGISTROS Y ANÁLISIS DE DATOS A GRAN ESCALA

Las **funciones de agregado** de SQL Server y otros motores de bases de datos relacionales realizan cálculos matemáticos a través de múltiples filas de una columna para devolver un **único valor de resumen** (Devart, 2026). El contrato fundamental de una función de agregación es '*muchas filas de entrada, una sola fila de salida*', lo que contrasta con las funciones escalares (una de entrada, una de salida) o las de ventana (muchas de entrada, muchas de salida) (SQL Practice Online, 2026). Estas funciones son la herramienta principal para la analítica empresarial y la generación de reportes estratégicos.

¡Peligro con los valores NULL en la agregación! Un error sumamente común es olvidar cómo se comportan las funciones ante campos nulos.

- **COUNT(*)** cuenta absolutamente todas las filas, incluyendo las que contienen valores NULL.
- **COUNT(columna)** cuenta únicamente las filas donde esa columna específica **no es NULL**.
- **SUM, AVG, MIN, MAX** ignoran por completo los valores nulos para sus cálculos. Por ejemplo, **AVG(calificación)** de {10, 20, NULL} dará como resultado 15, no 10, ya que el NULL se excluye del denominador. Para forzar un valor nulo a cero, use **COALESCE** o **ISNULL** (SQL Practice Online, 2026).

LAS 5 FUNCIONES DE AGREGADO UNIVERSALES

Función	Descripción y Propósito	Comportamiento ante NULL	Ejemplo Práctico de Sintaxis
COUNT(col / *)	Determina el número total de filas o registros en un conjunto de datos (Devart, 2026).	COUNT(*) cuenta filas con NULL. COUNT(col) ignora los valores NULL.	SELECT COUNT(*) FROM Ventas;
SUM(columna)	Suma todos los valores numéricos acumulados de una columna (Devart, 2026).	Ignora los campos NULL. Si todas las filas son NULL, devuelve NULL (no 0).	SELECT SUM(Monto) FROM Ventas;
AVG(columna)	Calcula la media aritmética (promedio) de una columna numérica (Devart, 2026).	Excluye los NULLs tanto del numerador como del denominador.	SELECT AVG(Monto) FROM Ventas;
MIN(columna)	Identifica el valor más pequeño en el conjunto de registros.	Ignora valores NULL. Funciona en números, cadenas de texto y fechas.	SELECT MIN(Monto) FROM Ventas;
MAX(columna)	Identifica el valor más grande en el conjunto de registros.	Ignora valores NULL. Funciona en números, cadenas de texto y fechas.	SELECT MAX(Monto) FROM Ventas;

AGRUPAMIENTO DE DATOS CON GROUP BY

La cláusula **GROUP BY** organiza las filas de una tabla en subgrupos lógicos basados en los valores comunes de una o más columnas (Devart, 2026). Las funciones de agregado calculan entonces sus métricas para cada grupo de forma independiente, en lugar de colapsar toda la tabla en un solo resultado (SQL Practice Online, 2026).

Regla Estricta de Oro: Cualquier columna individual que aparezca en el listado de SELECT y que no esté contenida dentro de una función de agregado **debe aparecer obligatoriamente** en la cláusula GROUP BY (SQL Practice Online, 2026; SQLabHub, 2026). De lo contrario, el motor de SQL rechazará la consulta por ambigüedad. Además, cabe destacar que **GROUP BY trata a los valores NULL como un grupo único**; todos los registros con valor nulo se colapsan en una sola categoría (SQL Practice Online, 2026).

FILTRADO DE GRUPOS CON LA CLÁUSULA HAVING

La cláusula **HAVING** se utiliza específicamente para filtrar los grupos resultantes creados por la cláusula **GROUP BY**. Mientras que **WHERE** actúa como un filtro inicial sobre las filas individuales, **HAVING** opera sobre los resultados ya agregados (Devart, 2026; SQLabHub, 2026). Debido a esto, **no es posible utilizar funciones de agregado dentro de la cláusula WHERE**; si se requiere aplicar una condición basada en una agregación (por ejemplo, mostrar departamentos cuyo promedio salarial sea superior a \$10,000), dicha condición debe declararse forzosamente en la cláusula **HAVING** (SQL Practice Online, 2026; SQLabHub, 2026).

COMPARATIVA EXHAUSTIVA: WHERE VS. HAVING

Característica	Cláusula WHERE	Cláusula HAVING
Propósito Principal	Filtra registros individuales (filas crudas) de la base de datos.	Filtra grupos consolidados después del agrupamiento.
Funciones de Agregado	¡Prohibidas! No permite funciones como SUM(), COUNT(), etc.	Diseñada específicamente para evaluar funciones de agregado.
Momento de Ejecución	Se ejecuta ANTES del agrupamiento (GROUP BY).	Se ejecuta DESPUÉS del agrupamiento (GROUP BY).
Uso de Índices	Altamente eficiente. El motor utiliza índices para descartar filas rápido.	Menos eficiente. Opera en memoria sobre datos pre-agregados.
Sintaxis e Independencia	Se puede usar sin GROUP BY en cualquier consulta simple.	Requiere lógicamente de GROUP BY (salvo pseudogrupo global).

ORDEN LÓGICO DE EJECUCIÓN DE UNA CONSULTA SQL (QUERY FLOW)

El motor de base de datos no procesa las consultas en el orden en que se escriben de izquierda a derecha. El flujo de proceso lógico sigue estrictamente la siguiente secuencia:

- 1. FROM / JOIN:** Reúne los datos de las tablas indicadas y aplica la condición de enlace ON.
- 2. WHERE:** Filtra las filas individuales que no cumplen la condición.
- 3. GROUP BY:** Divide las filas restantes en los subgrupos lógicos especificados.
- 4. HAVING:** Evalúa y descarta los grupos que no cumplen con las condiciones de agregación.
- 5. SELECT:** Define qué columnas se enviarán a la salida y evalúa alias y funciones escalares.
- 6. ORDER BY:** Ordena el conjunto de resultados final para su presentación (Ślawińska, 2022).

CASO PRÁCTICO APLICADO Y CÓDIGO SQL

Imaginemos una tabla de Ventas que registra transacciones del negocio. Deseamos obtener el total acumulado de ventas y el ticket promedio por vendedor, pero únicamente para aquellos vendedores que hayan realizado más de 1 venta en total. A continuación se modelan los datos de prueba donde se incluye a **Cristofer Garcia Luna** como vendedor destacado:

ID_Venta	Vendedor (Empleado)	Producto	Monto (USD)	Fecha_Venta
1	Cristofer Garcia Luna	Licencia BBDD	120.00	2026-08-10
2	Ana Martínez	Curso SQL	80.00	2026-08-11
3	Cristofer Garcia Luna	Soporte Técnico	150.00	2026-08-12
4	Diego Torres	Ebook Normalización	30.00	2026-08-13

```
-- Obtener ventas acumuladas y promedios por vendedor con más de 1 transacción
SELECT
  Vendedor,
  COUNT(*) AS Total_Transacciones,
  SUM(Monto) AS Ventas_Totales,
  AVG(Monto) AS Ticket_Promedio
FROM Ventas
WHERE Monto > 50.00 -- Filtro individual de transacciones importantes (WHERE)
GROUP BY Vendedor
HAVING COUNT(*) > 1; -- Filtro sobre grupos consolidados (HAVING)
```

Conclusión: Comprender la diferencia fundamental entre el filtrado fila a fila de la cláusula WHERE y el filtrado por conjuntos agregados de la cláusula HAVING es vital para el analista de datos. Esto permite construir consultas óptimas, evitando cálculos redundantes e innecesarios directamente en el servidor.

Fuentes Consultadas:

1. Sławińska, M. (2022). ¿Cuál es la diferencia entre las cláusulas WHERE y HAVING en SQL? LearnSQL.es.
2. Devart. (2026). Aggregate Functions in SQL Server: Full List, Syntax, and Use Cases. Devart Academy.